

DeepSeek-V4 in vLLM — An Implementation Study

A code-first walkthrough of how the DeepSeek-V4 architecture (paper [arXiv:2606.19348v1](#), §2, eqs 1–27) is implemented in **vLLM** ([vllm-project/vllm](#) @ commit [2b753ad20](#)). Chapter 1 gives the whole-package map; Chapters 2–11 open each component (mHC, CSA, HCA, the DeepSeek compressor, the lightning indexer, core MQA, output projection, KV-cache layout, MoE, MTP, quantization) with **diagrams**, **verbatim source snippets**, and **file paths** so you can read along in the repo.

Conventions. Paths are relative to [01_github/vllm/vllm/](#). Code is quoted verbatim from commit [2b753ad20](#) (line numbers may drift — a few lines of context are shown so blocks stay locatable). The **NVIDIA** backend is the reference path; AMD/XPU mirror the logic. Values tagged **[Flash]** come from the DeepSeek-V4-Flash config; **Pro** differs — always read the checkpoint [config.json](#).

DeepSeek-V4 in vLLM — An Implementation Study

1. Overview — how the model is organized in code

1.1 Package tree (annotated)

1.2 End-to-end forward

1.3 Class hierarchy & backend dispatch

2. mHC — Manifold-Constrained Hyper-Connections (§2.2, eqs 1–8)

3. DeepseekV4Attention — the MLA skeleton (attention.py)

3.1 Latent (MLA) projections

3.2 Layer-type dispatch (SWA / CSA / HCA)

4. The DeepSeek Compressor — CSA & HCA KV compression (compressor.py, eqs 9–12/20–23)

5. CSA lightning indexer — sparse top-k selection (sparse_attn_indexer.py, eqs 13–17)

6. HCA — Heavily Compressed Attention (eqs 20–26)

7. Core shared-KV MQA — FlashMLA (nvidia/flashmla.py, eqs 18–19/24–26)

8. Output projection, partial RoPE & attention sink (§2.3.3)

8.1 Grouped output projection

8.2 Partial + inverse RoPE

8.3 Attention sink (eq 27)

9. KV-cache layout & entries (sparse_mla.py, §2.3.4)

10. DeepSeekMoE feed-forward (§2.1)

11. MTP & quantization (§2.1, §2.3.4)

Appendix A — Paper equation → code index

Appendix B — Naming crib (paper ↔ vLLM ↔ ds4)

1. Overview — how the model is organized in code

DeepSeek-V4 lives in its **own package**, `vllm/models/deepseek_v4/`, registered in `vllm/model_executor/models/registry.py` as `"DeepseekV4ForCausalLM"`. The package holds the model graph and backend-specific kernels; it leans on **shared layers** elsewhere in the tree for mHC, the indexer, MLA, RoPE, and MoE routing.

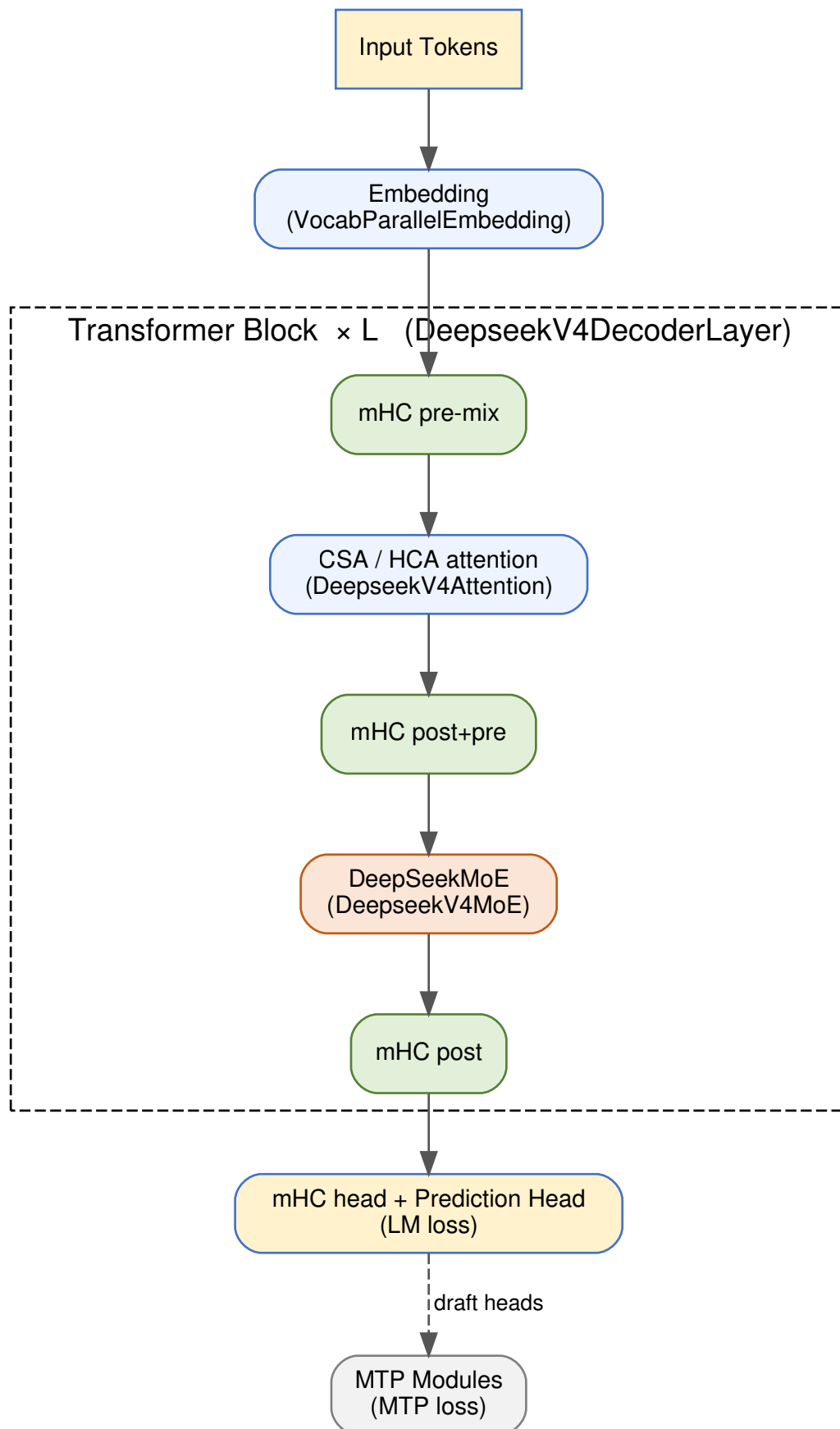
1.1 Package tree (annotated)

```
vllm/models/deepseek_v4/
├── __init__.py          # backend dispatch: is_rocm→amd, is_xpu→xpu, else nvidia
├── attention.py         # DeepseekV4Attention: MLA projections, layer-type dispatch
│                       #   (compress_ratio 1/4/128), wires Indexer + Compressor + SWA
├── cache
│   ├── compressor.py    # DeepseekCompressor: CSA (coff=2, overlap) / HCA (coff=1) KV
│   └── compression
│       ├── sparse_mla.py # KV-cache spec + fp8_ds_mla page layout (584 B/token), sparse MLA
│       └── metadata
│           ├── quant_config.py # expert dtype (fp4/fp8) + KV-cache dtype resolution
│           ├── common/
│           │   ├── rope.py      # build_deepseek_v4_rope (YaRN, partial last-64 RoPE)
│           │   └── ops/
│           │       ├── fused_triton_cutedsl.py # fused Triton/CuTe kernels: indexer_q, qk_rmsnorm,
│           │       │                       #   save_partial_states, inv_rope_fp8_quant, o_proj, ...
│           │       └── nvidia/
│           │           ├── model.py # DeepseekV4ForCausalLM / Model / DecoderLayer / MoE (reference
│           │           └── impl)
│           │               ├── flashmla.py # sparse FlashMLA core attention (prefill + decode)
│           │               ├── flashinfer_sparse.py, dspark.py, mtp.py
│           │               └── ops/sparse_attn_compress_cutedsl.py # CuTe-DSL compression pooling kernel
│           ├── amd/
│           │   ├── model.py, mtp.py, rocm.py
│           │   └── xpu/
│           │       ├── model.py, mtp.py, kernels
```

Shared layers this package depends on:

Concern	File
mHC math	<code>model_executor/kernels/mhc/torch.py</code> (+ <code>tilelang_kernels.py</code>), <code>model_executor/layers/mhc.py</code>
Lightning indexer	<code>model_executor/layers/sparse_attn_indexer.py</code>
MLA wrapper	<code>model_executor/layers/attention/mla_attention.py</code>
Partial/YaRN RoPE	<code>model_executor/layers/rotary_embedding/deepseek_scaling_rope.py</code>
MoE routing	<code>model_executor/layers/fused_moe/router/fused_topk_bias_router.py</code>
SWA cache	<code>v1/attention/backends/mla/sparse_swa.py</code>

1.2 End-to-end forward

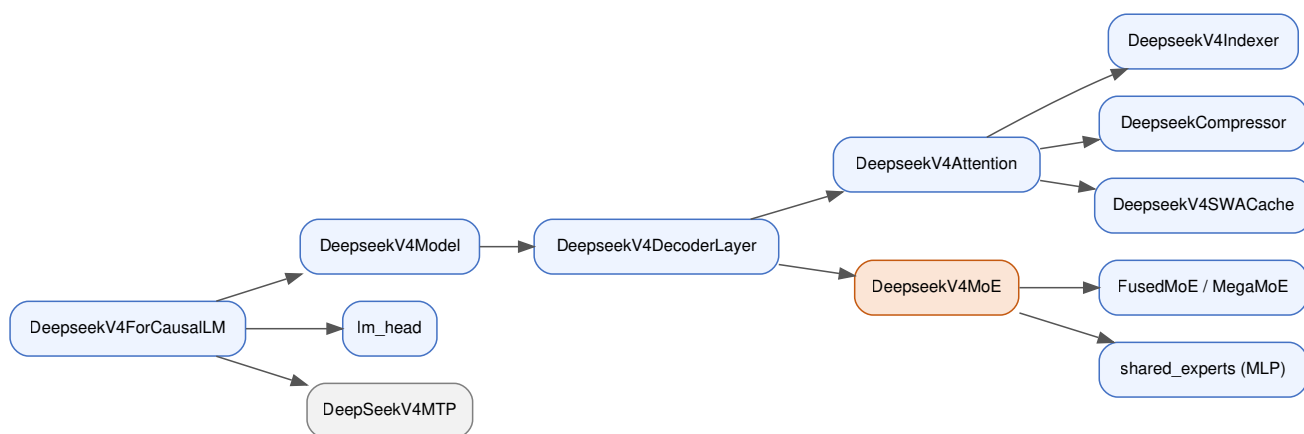


D1 · End-to-end forward (paper Fig. 2)

D1 · One decoder block = mHC-wrapped attention then mHC-wrapped MoE; $\times L$, then head + MTP. Compare to paper Figure 2.

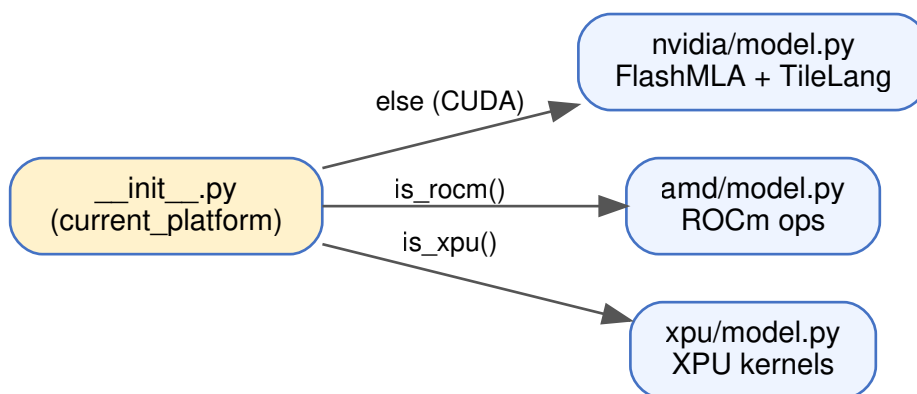
Each `DeepseekV4DecoderLayer` runs **mHC pre-mix** → **attention** → **mHC post+pre** → **MoE** → **mHC post**. The mHC "mixing" boxes replace the plain additive residual (Chapter 2). Attention is **CSA or HCA** depending on the layer (Chapters 3–7); the FFN is always **DeepSeekMoE** (Chapter 10).

1.3 Class hierarchy & backend dispatch



D2 · Class hierarchy (nvidia/model.py, attention.py)

D2 · Object graph (nvidia/model.py, attention.py).



D3 · Backend dispatch

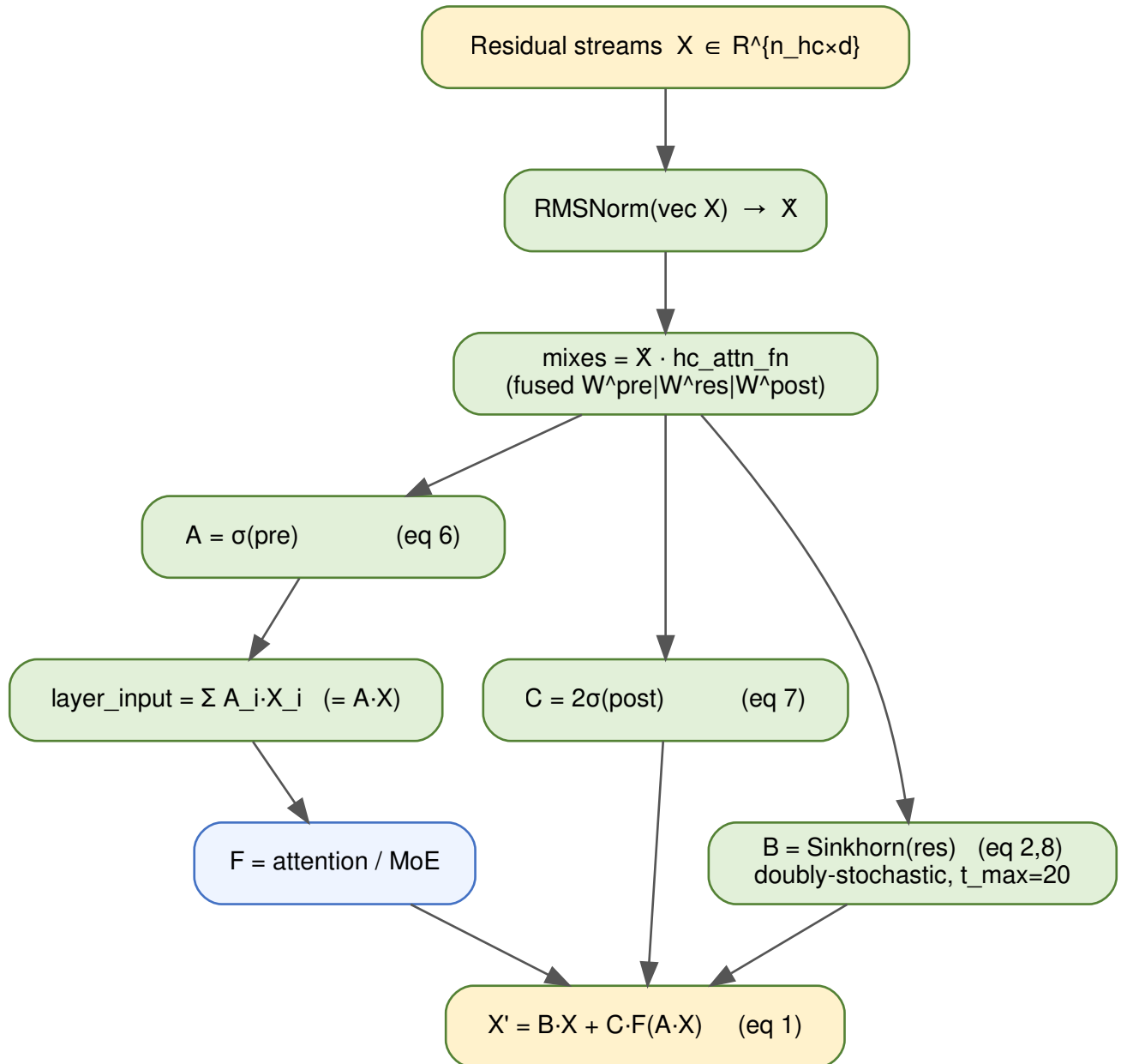
D3 · `__init__.py` selects the backend at import time.

```

# vllm/models/deepseek_v4/__init__.py
if current_platform.is_rocm():
    from .amd.model import DeepseekV4ForCausalLM
elif current_platform.is_xpu():
    from .xpu.model import DeepseekV4ForCausalLM
else: # NVIDIA (default)
    from .nvidia.model import DeepseekV4ForCausalLM
  
```

2. mHC — Manifold-Constrained Hyper-Connections (§2.2, eqs 1–8)

Instead of one residual vector, mHC carries **`n_hc` parallel residual streams** and mixes them with learned maps every layer; the stream-mixing matrix **`B`** is constrained to be **doubly stochastic** (non-expansive) for stability. Streams count = config **`hc_mult`**.



D4 · mHC stream mixing (kernels/mhc/torch.py)

D4 · mHC pre/post around a sublayer F (attention or MoE).

The reference math is [vllm/model_executor/kernels/mhc/torch.py](#). The **pre** op fuses RMSNorm + the three maps (eqs 3–7) and returns the layer input **`A·X`** (eq 1's inner term):

```

# vllm/model_executor/kernels/mhc/torch.py (mhc_pre_torch)
x = residual_flat.view(num_tokens, hc_mult * hidden_size).to(torch.float32)
mixes = torch.matmul(x, fn_flat.t()) #  $X \cdot [W^{pre}|W^{res}|W^{post}]$  (eqs 3-5)
sqrsum = x.square().sum(dim=-1, keepdim=True)
mixes = mixes * torch.rsqrt(sqrsum / (hc_mult * hidden_size) + rms_eps) #  $\leftarrow RMSNorm$  fused in

pre_logits = mixes[:, :hc_mult] * hc_scale[0] + hc_base[:hc_mult]
pre_mix = torch.sigmoid(pre_logits) + hc_pre_eps #  $A = \sigma(\tilde{A})$  (eq 6)
post_logits = mixes[:, hc_mult:2*hc_mult] * hc_scale[1] + hc_base[hc_mult:2*hc_mult]
post_mix = torch.sigmoid(post_logits) * hc_post_mult_value #  $C = 2 \cdot \sigma(C \sim)$  (eq 7)

comb_logits = mixes[:, 2*hc_mult:].view(num_tokens, hc_mult, hc_mult) * hc_scale[2] + hc_base[2*hc_mult:]...
comb_mix = torch.softmax(comb_logits, dim=-1) + hc_sinhorn_eps #  $M^{\theta} = \exp(B \sim)$  (eq 8)
comb_mix = comb_mix / (comb_mix.sum(dim=-2, keepdim=True) + hc_sinhorn_eps)
for _ in range(sinhorn_repeat - 1): # Sinkhorn-Knopp (eq 8)
    comb_mix = comb_mix / (comb_mix.sum(dim=-1, keepdim=True) + hc_sinhorn_eps) # row
    comb_mix = comb_mix / (comb_mix.sum(dim=-2, keepdim=True) + hc_sinhorn_eps) # col

layer_input = torch.sum(pre_mix.unsqueeze(-1) * residual_flat, dim=1) #  $A \cdot X$  (eq 1)

```

The **post** op reassembles the residual (eq 1's $B \cdot X + C \cdot F(\cdot)$):

```

# vllm/model_executor/kernels/mhc/torch.py (mhc_post_torch)
mixed_residual = torch.einsum("...ij,...ih->...jh", comb_res_mix, residual) #  $B \cdot X$ 
post_term = post_layer_mix * x.unsqueeze(-2) #  $C \cdot F(A \cdot X)$ 
return (mixed_residual + post_term) #  $X_{\{l+1\}}$  (eq 1)

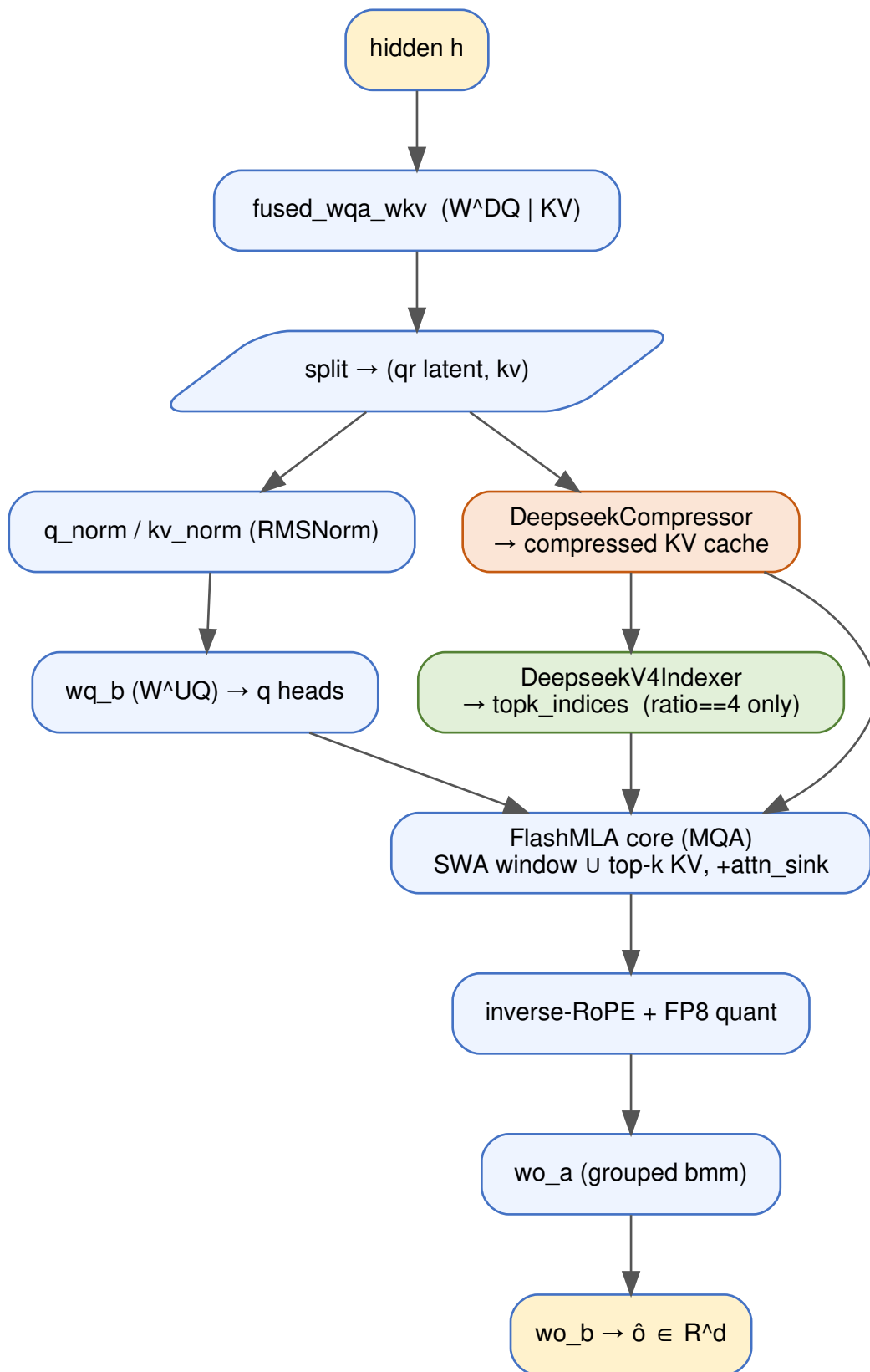
```

`sinhorn_repeat` = config `hc_sinhorn_iters` (paper `t_max = 20`); `hc_post_mult_value = 2.0`. The per-layer parameters `hc_attn_fn =  $[W^{pre}|W^{res}|W^{post}]$` , `hc_attn_base` (static biases S^*), and `hc_attn_scale` (gates α) are allocated on the decoder layer in `nvidia/model.py`.

Decoder forward threads the streams (`residual`, `post_mix`, `res_mix`) between layers instead of a single tensor: `mhc_pre` $\rightarrow$ `attn` $\rightarrow$ `mhc_fused_post_pre` $\rightarrow$ `moe` (`vllm/models/deepseek_v4/nvidia/model.py`, `DeepseekV4DecoderLayer.forward`). NVIDIA fuses these into `mhc_*_tilelang` kernels; the AMD path decomposes into `hc_pre` $\rightarrow$ `attn_norm` $\rightarrow$ `attn` $\rightarrow$ `hc_post`.

3. DeepseekV4Attention — the MLA skeleton (attention.py)

`vllm/models/deepseek_v4/attention.py` defines the shared attention body and dispatches the layer type.



D5 · DeepseekV4Attention forward dataflow (attention.py)

D5 · Attention dataflow: latent projections, compressor, indexer (CSA only), FlashMLA, grouped output.

3.1 Latent (MLA) projections

```
# vllm/models/deepseek_v4/attention.py (DeepseekV4Attention.__init__, ~199-235)
self.fused_wqa_wkv = MergedColumnParallelLinear(
    self.hidden_size, [self.q_lora_rank, self.head_dim], # → (q latent  $c_t^Q$ , kv latent)
    bias=False, quant_config=quant_config, prefix=f"{prefix}.fused_wqa_wkv", disable_tp=True)
self.q_norm = RMSNorm(self.q_lora_rank, self.eps)
self.wq_b = ColumnParallelLinear(self.q_lora_rank, self.n_heads * self.head_dim, ...) #  $W^{UQ} \rightarrow q \text{ heads}$ 
self.kv_norm = RMSNorm(self.head_dim, self.eps)
self.wo_a = ColumnParallelLinear(self.n_heads * self.head_dim // self.n_groups,
    self.n_groups * self.o_lora_rank, ...) # grouped output
(bmm)
self.wo_a.is_bmm = True; self.wo_a.bmm_batch_size = self.n_local_groups
self.wo_b = RowParallelLinear(self.n_groups * self.o_lora_rank, self.hidden_size, ...)
```

`fused_wqa_wkv` is the down-projection $W^{\{DQ\}}$ (produces the shared latent c_t^Q , eq 13/18) plus the KV latent. `wq_b` is the query up-projection $W^{\{UQ\}}$ (eq 18). `wo_a`/`wo_b` are the **grouped output** projection (Chapter 8).

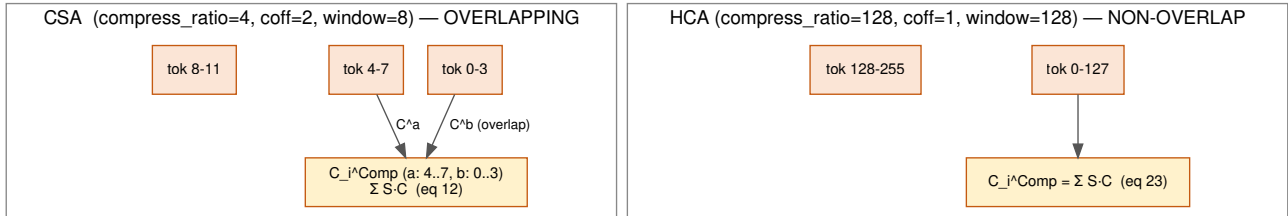
3.2 Layer-type dispatch (SWA / CSA / HCA)

```
# vllm/models/deepseek_v4/attention.py (~183-186)
if layer_id < config.num_hidden_layers:
    self.compress_ratio = max(1, config.compress_ratios[layer_id]) # 1=SWA, 4=CSA, 128=HCA
else:
    self.compress_ratio = 1
...
# (~248-268) only ratio-4 (CSA / "C4A") layers get the lightning indexer
self.indexer = None
if self.compress_ratio == 4:
    self.indexer = DeepseekV4Indexer(vllm_config, config=config, hidden_size=self.hidden_size,
    q_lora_rank=self.q_lora_rank, ..., compress_ratio=self.compress_ratio, prefix=f"{prefix}.indexer", ...)
```

A `DeepseekCompressor` is attached whenever `compress_ratio > 1` (CSA and HCA); SWA layers (ratio 1) keep the uncompressed sliding-window cache only. **[Flash]** the observed pattern is: layers 0–1 SWA, even layers CSA (ratio 4), odd layers HCA (ratio 128).

4. The DeepSeek Compressor — CSA & HCA KV compression (compressor.py, eqs 9–12/20–23)

`vllm/models/deepseek_v4/compressor.py` pools every m (CSA) or m' (HCA) raw tokens into one compressed KV entry via **softmax-weighted averaging** with a learnable positional bias.



D6 · DeepSeek compressor windows (compressor.py: `coff = 1 + (ratio==4)`)

D6 · CSA uses two overlapping windows (`coff=2`); HCA uses one non-overlapping window (`coff=1`).

The single switch that distinguishes CSA from HCA:

```
# vllm/models/deepseek_v4/compressor.py (~140-143)
assert compress_ratio in [4, 128]
coff = 1 + (compress_ratio == 4) # CSA: coff=2 (two windows a,b); HCA: coff=1
self.sliding_window = coff * compress_ratio # CSA: 8 (overlap 2·m); HCA: 128 (= m')
```

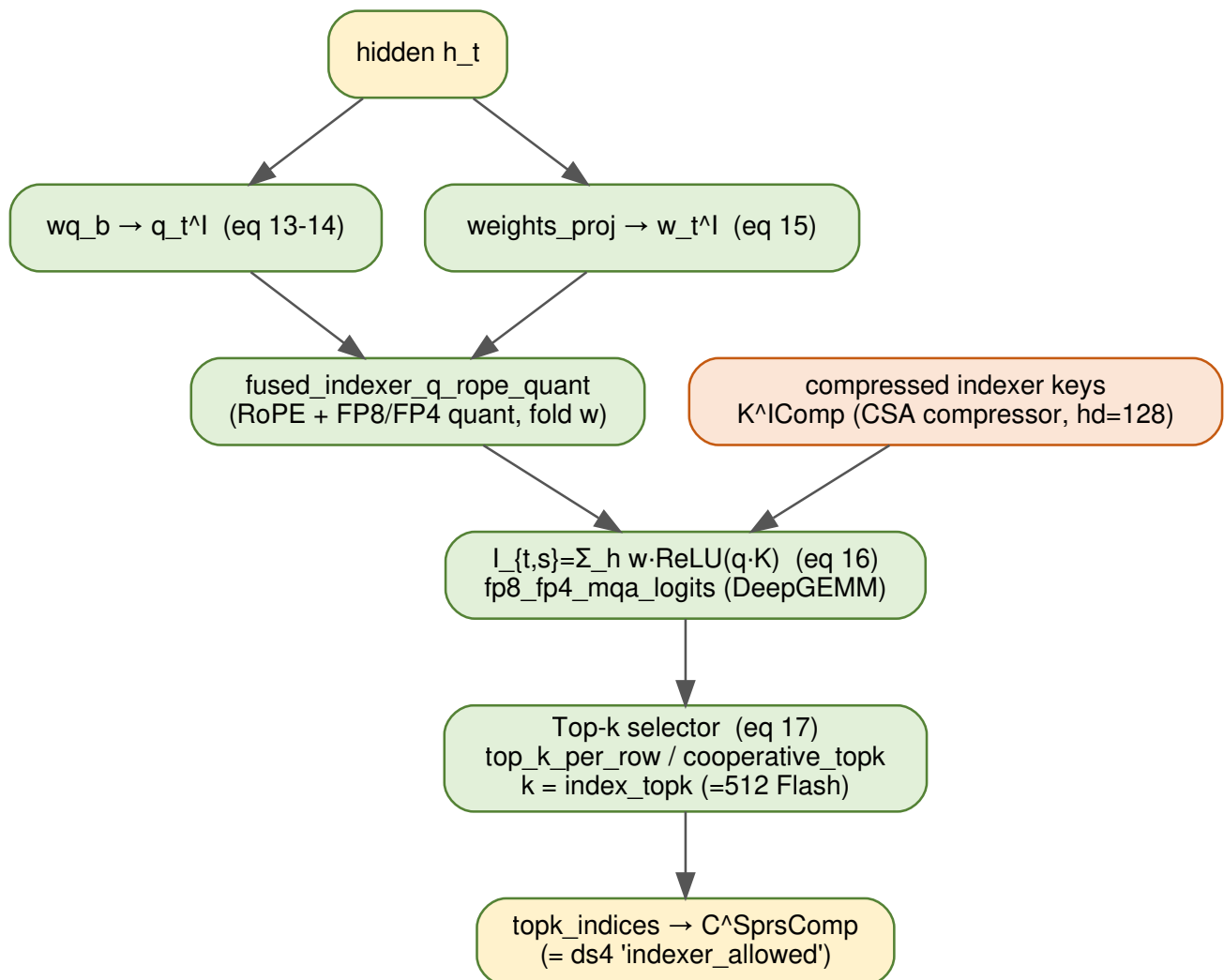
```
# vllm/models/deepseek_v4/compressor.py (~220-238)
self.ape = nn.Parameter(torch.empty((compress_ratio, self.coff * self.head_dim), dtype=torch.float32), ...)
self.fused_wkv_wgate = MergedColumnParallelLinear(
    self.hidden_size, [self.coff * self.head_dim, self.coff * self.head_dim], ...) # (C^a|C^b), (Z^a|Z^b)
self.norm = RMSNorm(self.head_dim, self.rms_norm_eps)
```

- `fused_wkv_wgate` produces the KV series C and the compression weights Z (eqs 9–10 for CSA, 20–21 for HCA). With `coff=2`, CSA packs the two overlapping series C^a, C^b / Z^a, Z^b into one matmul.
- `ape` = the learnable positional bias B added to Z before softmax; it is added per position (`ape[position % compress_ratio]`) inside `common/ops/save_partial_states.py`.
- The **softmax-weighted pooling** $C_i^{Comp} = \sum S_j \odot C_j$ (eqs 11–12 / 22–23) is done with a numerically-stable online (log-sum-exp) reduction in the CuTe-DSL kernel `vllm/models/deepseek_v4/nvidia/ops/sparse_attn_compress_cuteds.py` (a 128-wide variant handles HCA).

Net compression: CSA $1/m$ (overlapping 2m windows), HCA $1/m'$ (non-overlapping).

5. CSA lightning indexer — sparse top-k selection (sparse_attn_indexer.py, eqs 13–17)

This is the DSA "lightning indexer": it scores every compressed KV block against the query and keeps the **top-k**. Its output `topk_indices` is exactly the per-token selection our CPU study (`ds4`) traces.



D7 · CSA lightning indexer (layers/sparse_attn_indexer.py)

D7 · indexer query/keys → per-block scores → top-k → selected compressed KV.

Query + per-head weights (eqs 13–15). The indexer's `wq_b` up-projects the shared latent to indexer heads; `weights_proj` gives the per-head weight `w_t^I`. RoPE + FP8/FP4 quantization of Q is fused, folding `w · q_scale · softmax_scale` into the weights:

```
# vllm/model_executor/layers/sparse_attn_indexer.py (fused_indexer_q_rope_quant, ~209-250)
def fused_indexer_q_rope_quant(positions, q, cos_sin_cache, weights, softmax_scale,
    head_scale, is_neox):
    # RoPE + quantize Q to FP8; store weight * q_scale * softmax_scale * head_scale into
    weights_out
    ...
```

Scores $I_{\{t,s\}} = \sum_h w_h \cdot \text{ReLU}(q_h \cdot K_s^{\{\text{IComp}\}})$ (eq 16) — computed by DeepGEMM's fused MQA-logits kernel (ReLU + per-head weighting live *inside* the kernel):

```
# vllm/model_executor/layers/sparse_attn_indexer.py (prefill, ~466-473)
logits = fp8_fp4_mqa_logits(
    (q_slice_cast, q_scale_slice), (k_quant_cast, k_scale_cast),
    weights[chunk.token_start:chunk.token_end], # w_t^I
    cu_seqlen_ks, cu_seqlen_ke, clean_logits=False)
```

Top-k select (eq 17), $k = \text{config.index_topk}$ ([Flash] 512):

```
# vllm/model_executor/layers/sparse_attn_indexer.py (prefill ~475-484 / decode ~588-624)
ops.top_k_per_row_prefill(logits, cu_seqlen_ks, cu_seqlen_ke, topk_indices, num_rows,
                          logits.stride(0), logits.stride(1), topk_tokens) # topk_tokens =
k
# decode: torch.ops._C.cooperative_topk(...) / persistent_topk(...) /
top_k_per_row_decode(...)
```

The indexer keys $K^{\{\text{IComp}\}}$ come from a **CSA-style compressor with head_dim 128** (its own compressed KV cache), attached only on ratio-4 layers (Chapter 3.2).

Bridge to our experiment. `topk_indices` here $\equiv$ the boolean `indexer_allowed` / `sel[]` set that `ds4`'s `indexer_allowed_decode_one` logs on CPU — same selection mechanism, GPU kernels vs. a CPU reference loop. The temporal-locality study measures how stable this set is across decode steps.

6. HCA — Heavily Compressed Attention (eqs 20–26)

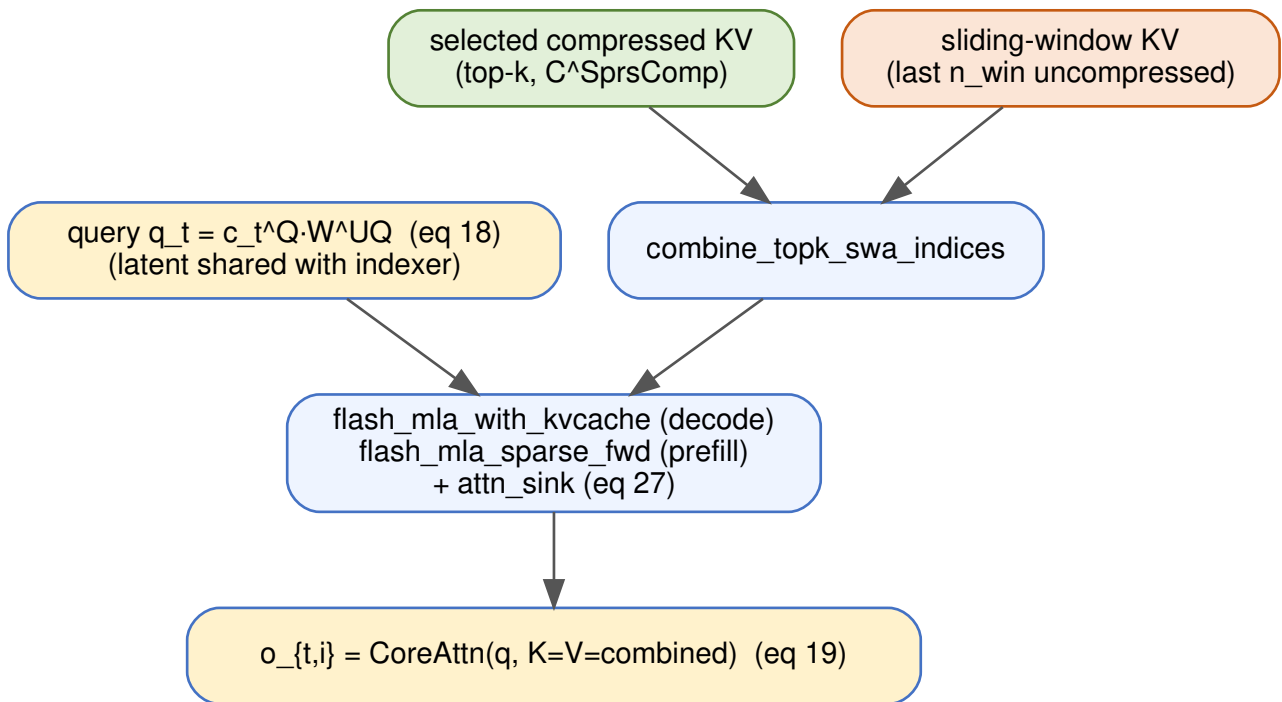
HCA is the cheap global path: **heavier compression** `m'` (=128), **no overlap, no sparse selection** — it attends densely over the (few) compressed entries.

- Same `DeepseekCompressor` with `coff = 1` (single `C`, `Z`; window = 128, non-overlapping). Pooling is the 128-wide CuTe kernel variant. Eqs 20–23.
- **No indexer** (`self.indexer is None` for ratio $\neq 4$), so no top-k — the core MQA runs over all `C^Comp`. Eqs 24–26 reuse the shared-KV MQA + grouped output of Chapters 7–8.

So CSA = "compress a little (1/4) + select top-k"; HCA = "compress a lot (1/128) + keep all". Interleaving them (plus the SWA branch) is what makes 1M-token context affordable.

7. Core shared-KV MQA — FlashMLA (nvidia/flashmla.py, eqs 18–19/24–26)

The core attention is **Multi-Query**: each compressed entry serves as *both* key and value; queries come from the shared latent. FlashMLA reads the **sliding-window KV** and the **top-k compressed KV** together.



D8 · Shared-KV MQA core (nvidia/flashmla.py)

D8 · SWA window $\cup$ top-k compressed KV $\rightarrow$ FlashMLA $\rightarrow$ o.

The latent query is up-projected once and shared with the indexer (eq 18): $q_t = c_t^Q \cdot W^{\{UQ\}} = wq_b(qr)$ (Chapter 3.1). Then the **decode** kernel:

```
# vllm/models/deepseek_v4/nvidia/flashmla.py (_forward_decode, ~219-235)
out, _ = flash_mla_with_kvcache(
    q=q, k_cache=swa_cache, head_dim_v=512, tile_scheduler_metadata=tile_metadata,
    is_fp8_kvcache=True,
    indices=swa_indices, topk_length=swa_lens,
    softmax_scale=self.scale, attn_sink=self.attn_sink,
    extra_k_cache=kv_cache if not swa_only else None,
    extra_indices_in_kvcache=topk_indices,
    extra_topk_length=topk_lens, out=output.unsqueeze(1))
# sliding-window KV
# attention sink (eq 27)
# compressed KV
# ← the indexer's top-k selection
```

Prefill first merges the two index sets, then runs the sparse kernel:

```
# vllm/models/deepseek_v4/nvidia/flashmla.py (_forward_prefill, ~326-347)
combined_indices, combined_lens = combine_topk_swa_indices(
    topk_indices[query_start:query_end], query_start_loc[...], seq_lens, gather_lens,
    self.window_size, self.compress_ratio, top_k, chunk_M, chunk_N)
flash_mla_sparse_fwd(
    q=q[query_start:query_end], kv=kv.view(-1, 1, q.shape[-1]),
    indices=combined_indices.unsqueeze(1), sm_scale=self.scale,
    attn_sink=self.attn_sink, topk_length=combined_lens, out=output[query_start:query_end])
```

The sliding-window cache is `DeepseekV4SWACache` (`v1/attention/backends/mla/sparse_swa.py`); it stores the last `n_win` **uncompressed** tokens, which are always attended alongside the selected compressed entries (paper §2.3.3 "additional branch of sliding window attention").

8. Output projection, partial RoPE & attention sink (§2.3.3)

8.1 Grouped output projection



D9 · Grouped output projection (attention.py wo_a/wo_b, ops/o_proj.py)

D9 · split heads into g groups → wo_a bmm per group → wo_b → hidden.

Because `c · n_h` is large, the output is projected group-wise: `wo_a` is a **batched matmul** over `n_groups` (`is_bmm=True`, Chapter 3.1), then `wo_b` maps to the hidden size. On NVIDIA this is fused as `deep_gemm_fp8_o_proj` in `vllm/models/deepseek_v4/nvidia/ops/o_proj.py`, which also applies the **inverse RoPE** before quantizing.

8.2 Partial + inverse RoPE

RoPE is applied to the **last 64 dims** only; the output carries an **inverse (position `-i`)** rotation so core-attention outputs encode *relative* position (paper §2.3.3):

```
# vllm/model_executor/layers/rotary_embedding/deepseek_scaling_rope.py (~263-306)
query_rot = query[..., -self.rotary_dim:]          # LAST rotary_dim (=64) dims
query_pass = query[..., :-self.rotary_dim]          # untouched leading dims
cos, sin = self.cos_sin_cache[positions].chunk(2, dim=-1)
if inverse:
    sin = -sin                                       # inverse RoPE (position -i)
query_rot = query_rot * cos + rotate_fn(query_rot) * sin
query = torch.cat((query_pass, query_rot), dim=-1)
```

The inverse pass on outputs is the fused kernel `common/ops/fused_inv_rope_fp8_quant.py` (`rope_dim=64, nope_dim=448`).

8.3 Attention sink (eq 27)

A per-head learnable logit `z'_h` is added to the softmax denominator, letting total attention drop below 1:

```
# vllm/models/deepseek_v4/attention.py (~194-197)
self.attn_sink = nn.Parameter(
    torch.full((self.padded_heads,), -float("inf"), dtype=torch.float32), requires_grad=False)
# passed to flash_mla_with_kvcache / flash_mla_sparse_fwd as attn_sink=self.attn_sink
```

It defaults to `-∞` (no sink) until the checkpoint fills the first `n_local_heads` slots.

9. KV-cache layout & entries (sparse_mla.py, §2.3.4)

The compressed KV uses DeepSeek's `fp8_ds_mla` paged format — a fixed **584-byte** slot per token:

fp8_ds_mla token slot = 584 B	448 B NoPE (FP8)
	128 B RoPE (BF16, 64 dims×2)
	8 B UE8M0 scale

paper §2.3.4: BF16 for RoPE dims,
FP8 for the rest
sparse_mla.py:106

D11 · KV cache entry layout (sparse_mla.py get_kv_cache_shape)

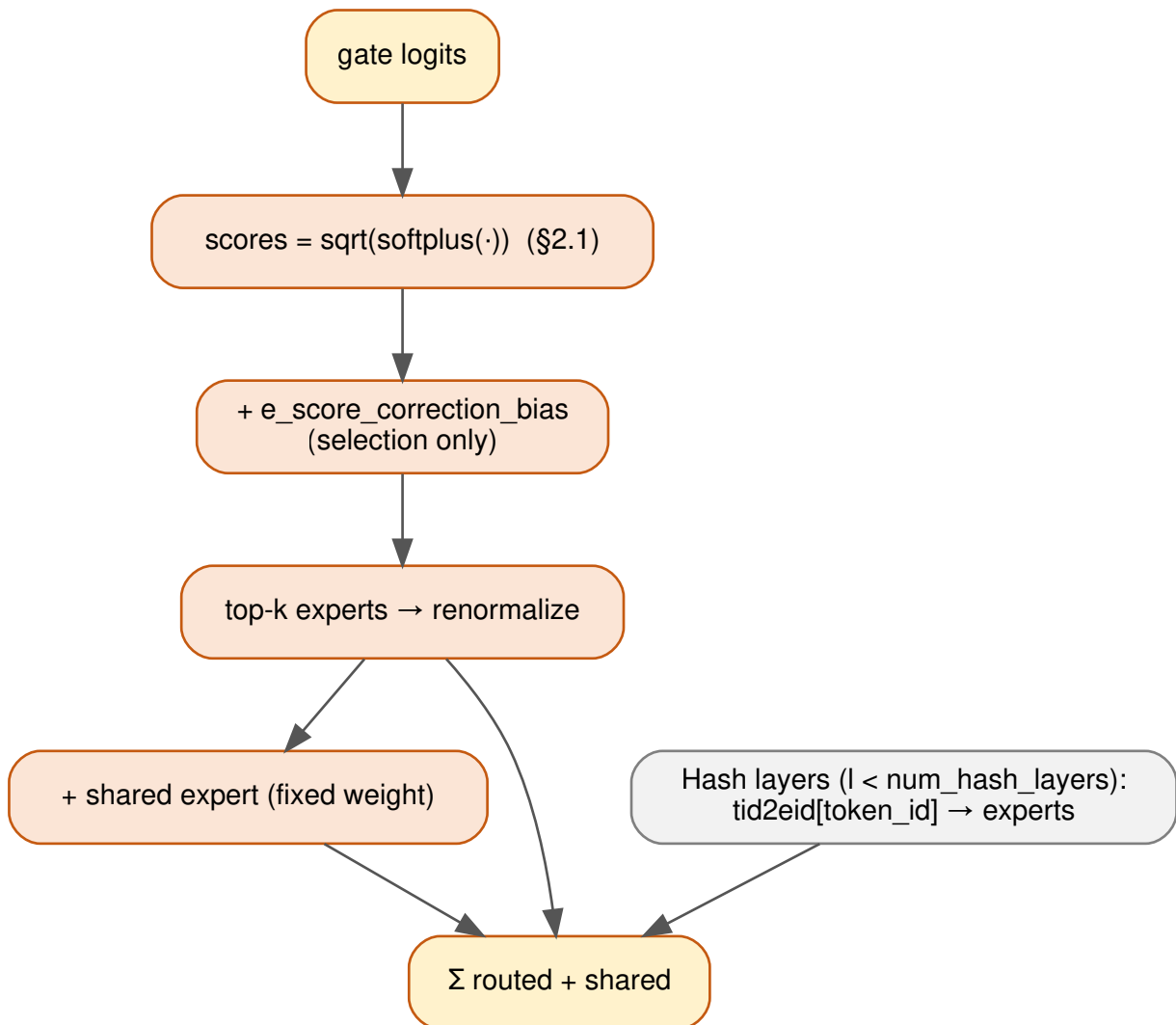
D11 · one fp8_ds_mla token slot = 448 B NoPE (FP8) + 128 B RoPE (BF16) + 8 B scale.

```
# vllm/models/deepseek_v4/sparse_mla.py (get_kv_cache_shape, ~98-108)
if cache_dtype_str == "fp8_ds_mla":
    # DeepseekV4 main MLA: 584B per token (448 NoPE + 128 RoPE + 8 fp8 scale).
    return (num_blocks, block_size, 584)
```

This is exactly the paper's "**BF16 for the RoPE dims, FP8 for the rest**": 448 NoPE dims in FP8 (1 B each) + 64 RoPE dims in BF16 (2 B each = 128 B) + an 8-B UE8M0 block scale. The KV **dtype** is resolved in `attention.py` (`_resolve_dsv4_kv_cache_dtype`), which forces the string `fp8_ds_mla` (stored as `uint8`) and drives the 576/584-B page specs. There are **two caches per attention layer**: the compressed CSA/HCA cache (this format) and the uncompressed `DeepseekV4SWACache` sliding window.

10. DeepSeekMoE feed-forward (§2.1)

The FFN is fine-grained DeepSeekMoE: **routed experts + shared experts**, with three V4 changes — `Sqrt(Softplus)` affinity, bias-for-selection load balancing, and **hash routing** in early layers.



D13 · DeepSeekMoE routing (fused_topk_bias_router.py)

D13 · sqrt-softplus scoring, bias-based top-k, renorm, shared expert; hash layers bypass scoring.

```
# vllm/model_executor/layers/fused_moe/router/fused_topk_bias_router.py (~59-102)
scores = torch.sqrt(F.softplus(gating_output.float())) # affinity = √softplus(·) (§2.1)
if e_score_correction_bias is not None:
    scores_for_choice = scores + e_score_correction_bias.float() # bias used for SELECTION only
# ... topk over scores for choice ...
weights = scores.gather(1, indices)
if renormalize:
    weights = weights / weights.sum(dim=-1, keepdim=True).clamp(min=1e-20)
# hash MoE: expert_ids = hash_indices_table[input_tokens] (no scoring)
```

- **Hash routing** in the first `num_hash_layers` blocks: a fixed table `tid2eid[token_id] → experts` (allocated in `nvidia/model.py`), so those layers skip scoring entirely.

- **Shared experts** (DeepseekV4MLP) are appended as fixed-weight slots after the routed top-k and added back to the output (`final_hidden_states += shared_output`). **[Flash]** 256 routed / 6 active + 1 shared.
-

11. MTP & quantization (§2.1, §2.3.4)

Multi-Token Prediction is unchanged from V3 — extra prediction modules used as speculative-decoding draft heads at inference:

```
# vllm/models/deepseek_v4/nvidia/mtp.py (~260-291)
class DeepSeekV4MTP(nn.Module):
    def __init__(self, *, vllm_config, prefix=""):
        self.model = DeepSeekV4MultiTokenPredictor(vllm_config=vllm_config, prefix=...)
    def forward(self, input_ids, positions, hidden_states, ...):
        return self.model(input_ids, positions, hidden_states, ...)
```

Quantization (`vllm/models/deepseek_v4/quant_config.py`): routed experts are **FP4** (MXFP4 or NVFP4, `expert_dtype="fp4"`) or **FP8 block** (`"fp8"`); attention linears use FP8 block scaling; the **indexer runs in FP4** (`common/ops/fused_indexer_q.py`, MXFP4 block 32). The `expert_dtype` property selects the MoE method (`Mxfp4MoEMethod` / `ModelOptNvFp4FusedMoE` / FP8), and `is_scale_e8m0` picks UE8M0 vs float32 linear scales.

Appendix A — Paper equation → code index

Paper	Code (file : symbol)
Fig 2 block / mHC-wrapped forward	<code>nvidia/model.py</code> <code>DeepseekV4DecoderLayer.forward</code>
mHC (1–8)	<code>model_executor/kernels/mhc/torch.py</code> <code>mhc_pre_torch</code> / <code>mhc_post_torch</code>
layer dispatch (m/m'/SWA)	<code>attention.py:183</code> <code>compress_ratios</code>
CSA compression (9–12)	<code>compressor.py</code> (<code>coff=2</code>) + <code>nvidia/ops/sparse_attn_compress_cutedsl.py</code>
lightning indexer + top-k (13–17)	<code>layers/sparse_attn_indexer.py</code> (<code>fp8_fp4_mqa_logits</code> , <code>*_topk</code>)
shared-KV MQA (18–19/24–26)	<code>nvidia/flashmla.py</code> + <code>layers/attention/mla_attention.py</code>
grouped output	<code>attention.py</code> <code>wo_a/wo_b</code> ; <code>nvidia/ops/o_proj.py</code>
HCA compression (20–23)	<code>compressor.py</code> (<code>coff=1</code>)
partial + inverse RoPE (§2.3.3)	<code>rotary_embedding/deepseek_scaling_rope.py</code> ; <code>common/ops/fused_inv_rope_fp8_quant.py</code>
attention sink (27)	<code>attention.py</code> <code>attn_sink</code>
KV <code>fp8_ds_mla</code> 584 B (§2.3.4)	<code>sparse_mla.py</code> <code>get_kv_cache_shape</code>
MoE sqrt-softplus + hash (§2.1)	<code>fused_moe/router/fused_topk_bias_router.py</code> ; <code>nvidia/model.py</code> <code>DeepseekV4MoE</code>
MTP	<code>nvidia/mtp.py</code> <code>DeepSeekV4MTP</code>

Appendix B — Naming crib (paper ↔ vLLM ↔ ds4)

paper	vLLM	ds4 (CPU)
CSA ratio <code>m = 4</code>	<code>compress_ratio==4, coff=2</code>	ratio-4 (even) layers
HCA ratio <code>m' = 128</code>	<code>compress_ratio==128, coff=1</code>	ratio-128 (odd) layers
lightning indexer top-k	<code>sparse_attn_indexer</code> → <code>topk_indices (index_topk)</code>	<code>indexer_allowed / sel</code>
mHC streams <code>n_hc</code>	<code>hc_mult</code>	—
SWA window <code>n_win</code>	<code>DeepseekV4SWACache.window_size</code>	<code>raw_kv_rows = 128</code>